



Design Pattern of the Week

Adapter

All things you should know about Adapter.



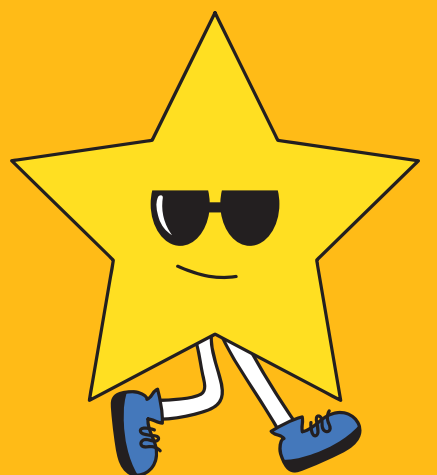
What?

It belongs to the structural
category.

It lets classes work together,
although they have
incompatible interfaces.



Shirin Monzavi



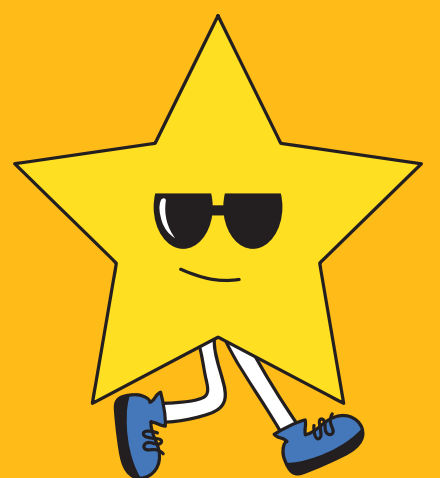


Problem

Consider for example a drawing editor which provides users draw and arrange graphical elements (lines, text, etc.). This editor provides an abstract class named Shape for all graphical objects. Classes for elementary shapes like LineShape and PolygonShape are easy to implement, but TextShape is not because it displays and edits text. Meanwhile, there is a toolkit class which already implements displaying and editing text named TextView, but it was not designed with Shape classes. Therefore, we cannot use TextView and Shape interchangeably. How can we cope with this issue?



Shirin Monzavi

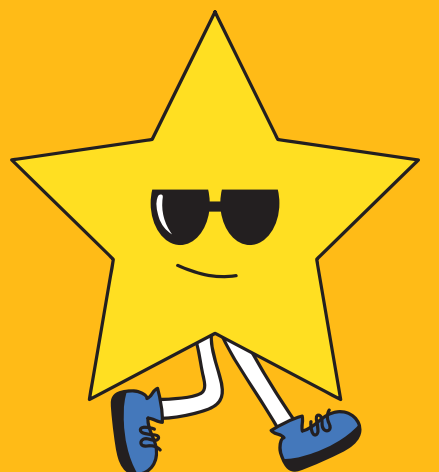


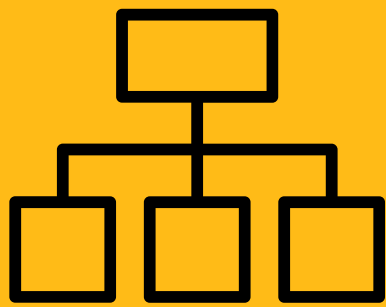


Solution

We can define TextShape so that it adapts the TextView to Shape.

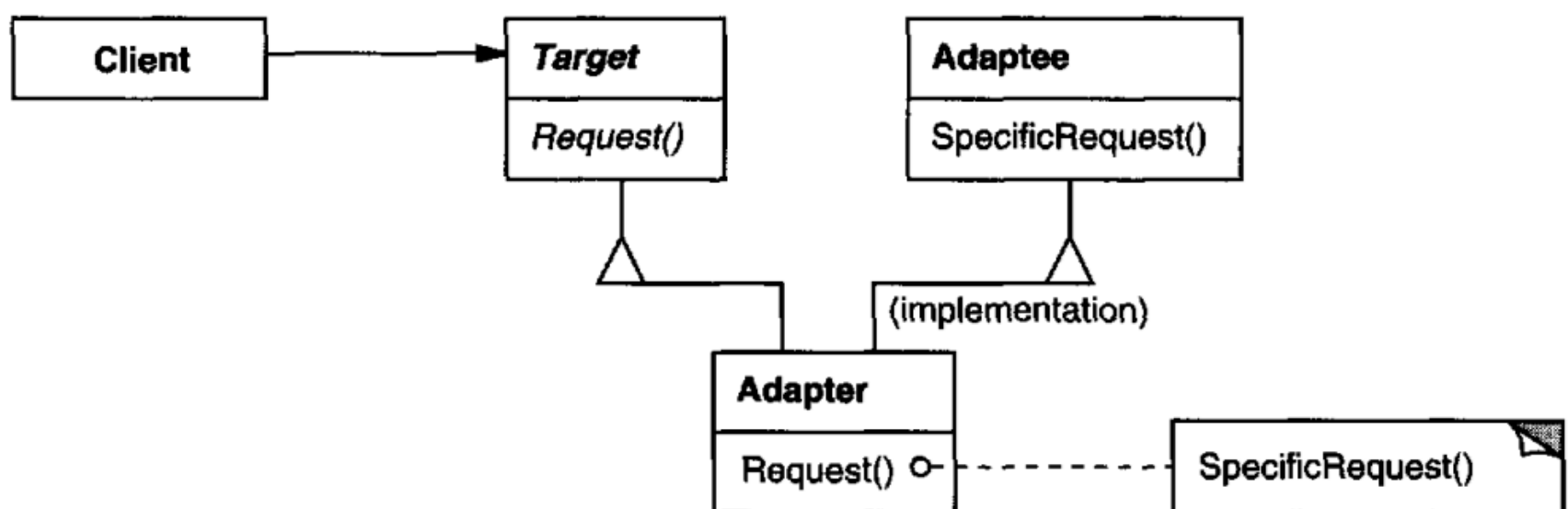
- 1- Inheriting Shape's interface and TextView class.
- 2- Composing TextView instance within TextShape.



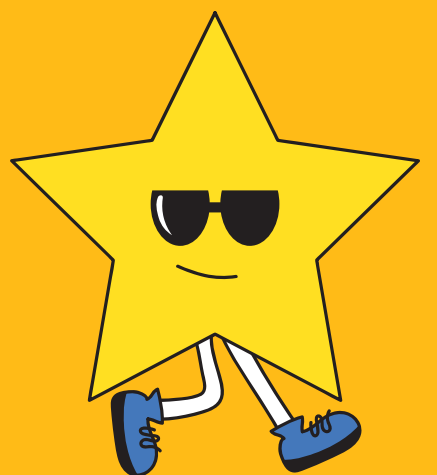
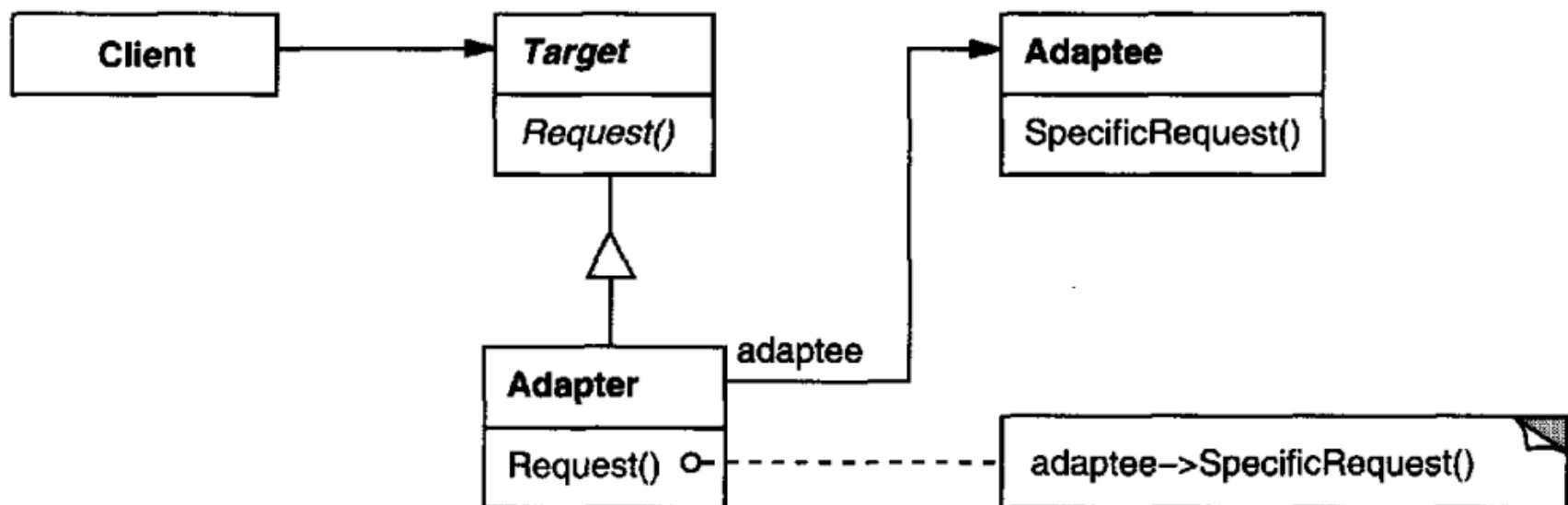


Structure

A class adapter uses multiple inheritance to adapt one interface to another:



An object adapter relies on object composition:



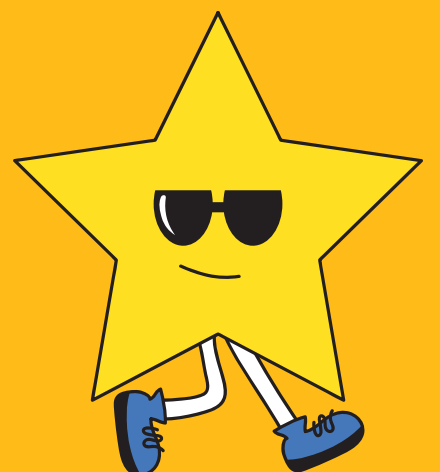


Usability

Use an existing class, and its interface **does not match** the one you need.

Create a reusable class that cooperates with unrelated or unforeseen classes, that is, classes that don't necessarily have compatible interfaces.

Use several existing subclasses, but it's unpractical to adapt their interface by subclassing every one. An object adapter can **adapt** the interface of **its parent class**.





● ● ● ● ● ● ● ● ● ● ● ● ● ● ● ●

? Have you ever used the Adapter Pattern in your projects?

☁ Share your experience in the comments!

● ● ● ● ● ● ● ● ● ● ● ● ● ● ● ●



Shirin Monzavi



follow

↕ Repost